

6.1 printf

`printf`'s name comes from **print** formatted. It generates output under the control of a *format string* (its first argument) which consists of literal characters to be printed and also special character sequences--*format specifiers*--which request that other arguments be fetched, formatted, and inserted into the string. Our very first program was nothing more than a call to `printf`, printing a constant string:

```
printf("Hello, world!\n");
```

Our second program also featured a call to `printf`:

```
printf("i is %d\n", i);
```

In that case, whenever `printf` "printed" the string "i is %d", it did not print it verbatim; it replaced the two characters %d with the value of the variable `i`.

There are quite a number of format specifiers for `printf`. Here are the basic ones :

%d	print an int argument in decimal
%ld	print a long int argument in decimal
%c	print a character
%s	print a string
%f	print a float or double argument
%e	same as %f, but use exponential notation
%g	use %e or %f, whichever is better
%o	print an int argument in octal (base 8)
%x	print an int argument in hexadecimal (base 16)
%%	print a single %

It is also possible to specify the width and precision of numbers and strings as they are inserted (somewhat like FORTRAN `format` statements); we'll present those details in a later chapter. (Very briefly, for those who are curious: a notation like %3d means to print an `int` in a field at least 3 spaces wide; a notation like %5.2f means to print a `float` or `double` in a field at least 5 spaces wide, with two places to the right of the decimal.)

To illustrate with a few more examples: the call

```
printf("%c %d %f %e %s %d%%\n", '1', 2, 3.14, 56000000., "eight", 9);
```

would print

```
1 2 3.140000 5.600000e+07 eight 9%
```

The call

```
printf("%d %o %x\n", 100, 100, 100);
```

would print

```
100 144 64
```

Successive calls to `printf` just build up the output a piece at a time, so the calls

```
printf("Hello, ");  
printf("world!\n");
```

would also print `Hello, world!` (on one line of output).

Earlier we learned that C represents characters internally as small integers corresponding to the characters' values in the machine's character set (typically ASCII). This means that there isn't really much difference between a character and an integer in C; most of the difference is in whether we choose to interpret an integer as an integer or a character. `printf` is one place where we get to make that choice: `%d` prints an integer value as a string of digits representing its decimal value, while `%c` prints the character corresponding to a character set value. So the lines

```
char c = 'A';  
int i = 97;  
printf("c = %c, i = %d\n", c, i);
```

would print `c` as the character `A` and `i` as the number `97`. But if, on the other hand, we called

```
printf("c = %d, i = %c\n", c, i);
```

we'd see the decimal value (printed by `%d`) of the character `'A'`, followed by the character (whatever it is) which happens to have the decimal value `97`.

You have to be careful when calling `printf`. It has no way of knowing how many arguments you've passed it or what their types are other than by looking for the format specifiers in the format string. If there are more format specifiers (that is, more `%` signs) than there are arguments, or if the arguments have the wrong types for the format specifiers, `printf` can misbehave badly, often printing nonsense numbers or (even worse) numbers which mislead you into thinking that some other part of your program is broken.

Because of some automatic conversion rules which we haven't covered yet, you have a small amount of latitude in the types of the expressions you pass as arguments to `printf`. The argument for `%c` may be of type `char` or `int`, and the argument for `%d` may be of type `char` or `int`. The string argument for `%s` may be a string constant, an array of characters, or a pointer to some characters (though we haven't really covered strings or pointers yet). Finally, the arguments corresponding to `%e`, `%f`, and `%g` may be of types `float` or `double`. But other combinations do *not* work reliably: `%d` will not print a long `int` or a `float` or a `double`; `%ld` will not print an `int`; `%e`, `%f`, and `%g` will not print an `int`.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)